

Lab [Option C - Java]: CodeCraftHub Learning Management System - Building a Personalized Learning Platform



Estimated time: 1.5 hours

Welcome to this final project lab. Your organization is creating CodeCraftHub, a personalized learning platform crafted for developers! Your task is to design the server-side architecture for this learning platform and develop the server-side application using Java and Spring Boot. You will create the server-side components for CodeCraftHub using Generative AI (GenAI).

In this project, you'll leverage the power of GenAI to transform your vision into a reality.

Learning objectives

After completing this lab, you will be able to perform the following tasks:

- Design and develop software applications using GenAI
- Create documentation for the code with GenAI
- Create test cases with GenAI
- Run and test the application developed with GenAI

Prerequisites

- Proficiency in Java
- An active GitHub account
- Comfort working with a local IDE
- Working knowledge of curl

Setting up the AI classroom

As part of this assignment, you will need to set up the GenAI classroom. If you need assistance, please refer to the following instructions: [Get Familiar with GenAI Classroom](#)

Gathering requirements for development of the learning platform

When using GenAI to gather your requirements for developing the server-side learning platform, keep the following considerations in mind:

- The effectiveness of the responses depends on the prompts provided.
- The prompts provided here are suggestions; use your discretion to adapt them to your needs.
- Apply your subject matter expertise and judgment as a developer.
- You are responsible for validating the correctness of the responses.

Note: In the GenAI interface, select your preferred LLM.

Enter the following prompt to provide the LLM with context and define the objective:



plaintext



I want to create a simple personalized learning platform called CodeCraftHub

Requirements:

- Use Java with Spring Boot framework
- Store course data in a simple JSON text file (NO database needed)
- No authentication or user management needed
- Focus on learning REST API basics

Each course should track:

- Course name
- Description
- Target completion date
- Current status (Not Started, In Progress, Completed)

Recommend:

1. A simple project structure for beginners
2. The REST API endpoints to be created
3. How to store and retrieve data from a JSON file

► [Click here to view the sample response generated](#)

Disclaimer: The response generated by your LLM might differ from the sample response shown here.

Creating the application structure

For the following exercise, a simple Spring Boot application with basic layered architecture is the recommended approach.

In the GenAI classroom, in the **type your prompt field** enter the following prompt to create the main application file:



Now, create the complete Java Spring Boot code with these requirements:

1. Create a Spring Boot REST API with all CRUD operations for courses
2. Store data in a JSON file called "courses.json"
3. Include these endpoints:
 - POST /api/courses - Add a new course
 - GET /api/courses - Get all courses
 - GET /api/courses/{id} - Get a specific course
 - PUT /api/courses/{id} - Update a course
 - DELETE /api/courses/{id} - Delete a course
4. Each course must have these JSON fields:
 - id (auto-generated, starting from 1)
 - name (required)
 - description (required)
 - target_date (required in JSON requests, format YYYY-MM-DD)
 - status (required, must be exactly one of: "Not Started", "In Progress",
 - created_at (auto-generated timestamp in JSON responses)

In the Java model, you may use camelCase field names such as `targetDate``

5. Include proper error handling for:
 - Missing required fields
 - Course not found
 - Invalid status values
 - File read/write errors
6. Add helpful comments throughout the code for beginners
7. Make sure the app creates courses.json automatically if it doesn't exist
8. Use Jackson for JSON processing

Please provide:

- Course.java (model class)
- CourseService.java (service layer with file operations)
- CourseController.java (REST controller)
- CodeCraftHubApplication.java (main class)
- pom.xml (Maven dependencies)

The response will include the complete Spring Boot application code.

Note: Use `snake_case` for JSON request and response fields, such as `target_date` and `created_at`. In the Java model class, `camelCase` field names such as `targetDate` and `createdAt` are acceptable when they are mapped with Jackson annotations. Use the status values exactly as shown: `Not Started`, `In Progress`, and `Completed`.

► [Click here to view the sample response generated](#)

Disclaimer: The response generated by your LLM might differ from the sample response shown here.

Generate README documentation

After you generate your application code, use GenAI to create project documentation.

Enter the following prompt to create a `README.md` file for the CodeCraftHub project:



plaintext



Create a `README.md` file for the CodeCraftHub project that includes:

1. Project overview and features
2. Installation instructions
3. How to run the application
4. API endpoint documentation with examples
5. Troubleshooting guide

► [Click here to view the sample response generated](#)

Disclaimer: The response generated by your LLM might differ from the sample response shown here.

Running and testing your application on your local machine

Complete the following steps to run and test your application on your local machine.

Step 1: Set up your development environment

Prerequisites

Before you run the Maven commands, make sure the following software is installed on your machine:

Required Software

Java Development Kit (JDK): Minimum version: JDK 17 (for most modern Spring Boot 3.x applications) Recommended versions: JDK 17 or JDK 21 (LTS versions)

Apache Maven: Minimum version: Maven 3.6.3 or higher Recommended: Maven 3.8+

1. Create a new project directory:

 bash 

```
mkdir codecrafthub
cd codecrafthub
```

 Run

1. Create the project directory structure:

 bash 

```
mkdir -p src/main/java/com/codecrafthub/controller
mkdir -p src/main/java/com/codecrafthub/model
mkdir -p src/main/java/com/codecrafthub/service
mkdir -p src/main/resources
```

 Run

1. Create your Java files and `pom.xml` by using the code generated by GenAI.

Step 2: Build the project

 bash 

```
mvn clean install
```

 Run

Step 3: Run the application

 bash 

```
mvn spring-boot:run
```

 Run

You should see something similar:

 plaintext 

```
- CodeCraftHub API is starting...  
- Data will be stored in: courses.json  
- API will be available at: http://localhost:8080
```

Step 4: Test the API endpoints

Make sure the server is still running, as shown in Step 3.

In the Generative AI classroom, in the **type your prompt** field enter the following prompts one at a time to generate the curl commands you will use for testing. Then, run each generated command in a new terminal window.

The prompts appear in plain code blocks so you can distinguish them from executable terminal commands. Executable commands appear in `bash` code blocks.

Prompt 1: Generate a POST command

 yaml 

```
Create a curl command for a POST request to http://localhost:8080/api/courses
- name: Python Basics
- description: Learn Python fundamentals
- target_date: 2025-12-31
- status: Not Started

Return only the curl command.
```

Sample command:

Run a command similar to the following in a new terminal window:

 bash 

```
curl -X POST http://localhost:8080/api/courses \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Python Basics",
    "description": "Learn Python fundamentals",
    "target_date": "2025-12-31",
    "status": "Not Started"
  }'
```

Run

Prompt 2: Generate GET commands

 plaintext 

```
Create two curl commands for this API:
1. A GET request to http://localhost:8080/api/courses to retrieve all courses
2. A GET request to http://localhost:8080/api/courses/1 to retrieve the cours

Return only the curl commands.
```

Sample commands:

bash

```
curl http://localhost:8080/api/courses
curl http://localhost:8080/api/courses/1
```

Run

Prompt 3: Generate a PUT command

plaintext

Create a curl command for a PUT request to `http://localhost:8080/api/courses/`

Return only the curl command.

Sample command:

Run a command similar to the following in a new terminal window:

bash

```
curl -X PUT http://localhost:8080/api/courses/1 \
  -H "Content-Type: application/json" \
  -d '{
    "status": "In Progress"
  }'
```

Run

Prompt 4: Generate a DELETE command

plaintext

Create a curl command for a DELETE request to `http://localhost:8080/api/cours`

Return only the curl command.

Sample command:



bash



```
curl -X DELETE http://localhost:8080/api/courses/1
```

▶ Run

Verify that:

- All endpoints return appropriate responses
- Data persists in `courses.json`
- Error handling works correctly

[Optional] Challenge

If you complete the basic requirements, consider adding this bonus feature using GenAI:

Enhancement: Statistics Endpoint

- Show the total number of courses by status

Prompt hint:

 plaintext 

```
Add a new endpoint GET /api/courses/stats that returns statistics about cours
- Total number of courses
- Number of courses by status (Not Started, In Progress, Completed)

Add this to the CourseController.java file.
```

Pushing your work to GitHub

Remember, any modifications you make in the lab environment won't be saved. If you plan to step away, use the following steps to make sure your changes are pushed to GitHub:

Verify that you are currently inside the project directory.

1. Navigate to the project directory by performing `cd codecrafthub`.
2. Set up your Git configuration:
 - Run: `git config --global user.email "yourgithub@email.com"`
 - Run: `git config --global user.name "name"`
3. Initialize the repository (if not already done):
 - Run: `git init`
4. Add your changes to the staging area:
 - Run: `git add .`
5. Commit your changes with a descriptive message:
 - Run: `git commit -m "Complete CodeCraftHub learning platform"`
6. The first step is to generate an access token from GitHub.com. Follow the lab named [Generate GitHub personal access token](#) and copy the access token as a password in the upcoming exercises.
7. Create a new repository on GitHub.com and copy the repository URL.
8. Link your local repository to GitHub:
 - Run: `git remote add origin <your-github-repo-url>`

- Run: `git push -u origin main`

10. An on-screen message in the terminal window will prompt you to enter your GitHub username and password (your previously created "Personal Access Token" in Step 6).

These steps verify that you safely stored your project in GitHub, allowing you to continue.

Exemplar code

For your reference, select the following link to download a zip file that provides an example code. After downloading the file, unzip the file in your local machine.

[CodeCraftHub code \[Java\]](#)

Reviewing your checklist

At this stage, you have completed the following tasks:

- You have a running application that offers REST API functionality for the CodeCraftHub learning platform.
- You created a simple, file-based storage system using JSON.
- The application includes proper error handling and validation.
- You documented your code and created test cases.
- [Optional] You pushed all the code to your GitHub repository.

Summary

You've now achieved the following accomplishments:

- Successfully gathered requirements for a simple learning platform using GenAI
- Created a complete Spring Boot REST API with CRUD operations
- Implemented file-based data storage using JSON
- Created comprehensive test cases and documentation
- Validated that your application works correctly

Congratulations! You have successfully leveraged Generative AI to build the CodeCraftHub Learning Management System using Java and Spring Boot.

Author(s)

[Ramanujam Srinivasan](#)

© IBM Corporation. All rights reserved.

Author(s)

[Ramanujam Srinivasan](#)

© IBM Corporation. All rights reserved

Lab [Option C - Java]: CodeCraftHub Learning Management System - Building a Personalized Learning Platform



Estimated time: 1.5 hours

Welcome to this final project lab. Your organization is creating CodeCraftHub, a personalized learning platform crafted for developers! Your task is to design the server-side architecture for this learning platform and develop the server-side application using Java and Spring Boot. You will create the server-side components for CodeCraftHub using Generative AI (GenAI).

In this project, you'll leverage the power of GenAI to transform your vision into a reality.

Learning objectives

After completing this lab, you will be able to perform the following tasks:

- Design and develop software applications using GenAI
- Create documentation for the code with GenAI
- Create test cases with GenAI
- Run and test the application developed with GenAI

Prerequisites

- Proficiency in Java
- An active GitHub account
- Comfort working with a local IDE
- Working knowledge of curl

Setting up the AI classroom

As part of this assignment, you will need to set up the GenAI classroom. If you need assistance, please refer to the following instructions: [Get Familiar with GenAI Classroom](#)

Gathering requirements f... →